

Código-Fonte _ InsurMinds _ Projeto _ Final_IA2A_2026.

Gerar_apolices_mvp.py

Baseado nas informações dados reais das condições de apólices foram criadas duas apólices:

- Todos os dados condições gerais são reais extraídas dados seguradoras.
- Os nomes dos segurador e seus dados são fakes, serviram apenas para o aprendizado do trabalho.



Script(1): Gerar_apolices_mvp.py

```
import os
```

```
from reportlab.lib.pagesizes import letter
```

```
from reportlab.platypus import SimpleDocTemplate, Paragraph, Spacer, Table, TableStyle
```

```
from reportlab.lib.styles import getSampleStyleSheet, ParagraphStyle
```

```
from reportlab.lib import colors
```

Função mestre que monta o layout e o texto em PDF da apólice

```
def criar_pdf_apolice(nome_arquivo, seguradora, cnpj_seg, processo_susep, num_apolice, lmi_valor,
    franquia_b_valor, def_texto, reclamacao_texto):
```

```
    doc = SimpleDocTemplate(nome_arquivo, pagesize=letter, rightMargin=36, leftMargin=36, topMargin=36,
        bottomMargin=36)
```

```
    story = []
```

```
    styles = getSampleStyleSheet()
```

```

# Definição dos estilos visuais do documento
style_header = ParagraphStyle('H1', fontName='Helvetica-Bold', fontSize=12, leading=14,
alignment=1, spaceAfter=10)
style_section = ParagraphStyle('H2', fontName='Helvetica-Bold', fontSize=10, leading=12,
spaceBefore=12, spaceAfter=6, textColor=colors.HexColor('#005A9C'))
style_body = ParagraphStyle('Body', fontName='Helvetica', fontSize=9, leading=12, spaceAfter=4)
style_table_header = ParagraphStyle('TH', fontName='Helvetica-Bold', fontSize=9, leading=11,
textColor=colors.white)

# Cabeçalho regulatório da SUSEP
story.append(Paragraph("<b>FRONTISPÍCIO DE APÓLICE DIGITAL – RISK MANAGEMENT EXECUTIVE (D&O)
</b>", style_header))
story.append(Paragraph(f"<b>SEGURADORA:</b> {seguradora} | <b>CNPJ:</b> {cnpj_seg}",
style_body))
story.append(Paragraph(f"<b>REGISTRO PRODUTO SUSEP:</b> {processo_susep} | <b>Nº APÓLICE:</b>
{num_apolice}", style_body))
story.append(Spacer(1, 8))

# Tabela com dados fictícios do cliente/tomador
dados_contrato = [
    [Paragraph('<b>DADOS DO TOMADOR (EMPRESA):</b>', style_body), Paragraph('<b>DADOS DO
SEGURADO:</b>', style_body)],
    [Paragraph('ALFA METALOGRAFIA E TECNOLOGIA LTDA<br/>CNPJ: 99.999.999/0001-99<br/>SÃO PAULO -
SP', style_body),
    Paragraph('DIRETORES, CONSELHEIROS E<br/>ADMINISTRADORES ELEITOS DA TOMADOR', style_body)],
    [Paragraph('<b>VIGÊNCIA DA APÓLICE:</b>', style_body), Paragraph('<b>ÂMBITO GEOGRÁFICO:
</b>', style_body)],
    [Paragraph('13/09/2026 a 13/09/2027', style_body), Paragraph('Território Nacional e
Internacional', style_body)]
]

t_dados = Table(dados_contrato, colWidths=[270, 270])
t_dados.setStyle(TableStyle([
    ('BOX', (0,0), (-1,-1), 1, colors.HexColor('#005A9C')),
    ('BACKGROUND', (0,0), (-1,0), colors.HexColor('#E6F0FA')),
    ('BACKGROUND', (0,2), (-1,2), colors.HexColor('#E6F0FA')),
    ('GRID', (0,0), (-1,-1), 0.5, colors.lightgrey),
    ('VALIGN', (0,0), (-1,-1), 'TOP'),
]))
story.append(t_dados)

# Quadro técnico de coberturas e franquias (Onde a IA vai ler)
story.append(Paragraph("GARANTIAS CONTRATADAS, LIMITES E RETENÇÕES (FRANQUIAS)", style_section))
tabela_valores = [
    [Paragraph('<b>Modalidade / Cobertura</b>', style_table_header), Paragraph('<b>Limite Máximo
(LMI)</b>', style_table_header), Paragraph('<b>Franquia (Retenção Co. A)</b>',

```

```

style_table_header), Paragraph('<b>Franquia (Reembolso Co. B)</b>', style_table_header)],
    [Paragraph('Cobertura A - Direta Executivos', style_body), Paragraph(f"R$ {lmi_valor:,.2f}",
style_body), Paragraph('ISENTO (R$ 0,00)', style_body), Paragraph('Não Aplicável', style_body)],
    [Paragraph('Cobertura B - Reembolso Companhia', style_body), Paragraph(f"R$
{lmi_valor:,.2f}", style_body), Paragraph('Não Aplicável', style_body), Paragraph(f"R$
{franquia_b_valor:,.2f}", style_body)],
    [Paragraph('Cobertura Adicional - Custos de Defesa', style_body), Paragraph(def_texto,
style_body), Paragraph('ISENTO (R$ 0,00)', style_body), Paragraph('ISENTO (R$ 0,00)',
style_body)]
]

t_valores = Table(tabela_valores, colWidths=[140, 130, 130, 140])
t_valores.setStyle(TableStyle([
    ('BOX', (0,0), (-1,-1), 1, colors.HexColor('#005A9C')),
    ('BACKGROUND', (0,0), (-1,0), colors.HexColor('#005A9C')),
    ('GRID', (0,0), (-1,-1), 0.5, colors.lightgrey),
    ('VALIGN', (0,0), (-1,-1), 'MIDDLE'),
]))
story.append(t_valores)

# Cláusulas contratuais da apólice
story.append(Paragraph("OBJETO E CONDIÇÕES CONTRATUAIS GERAIS DO RISCO", style_section))
story.append(Paragraph("<b>1. RISCOS COBERTOS:</b> Este seguro de Responsabilidade Civil D&O
garante a indenização aos Segurados por Prejuízos decorrentes de reclamações de terceiros
baseadas estritamente em atos culposos de gestão (negligência, imperícia ou omissão) praticados
no exercício de suas funções administrativas capitaneadas no contrato social.", style_body))
story.append(Paragraph("<b>2. EXCLUSÃO DE DOLO E FRAUDE:</b> Estão formalmente excluídos da
cobertura securitária todos os atos tipificados como dolosos, fraudulentos ou que tragam
vantagens financeiras ilícitas aos administradores. O adiantamento de custos de defesa será
mantido até a sentença judicial Transitada em Julgado. Configurado o dolo final, haverá a
reversão imediata com obrigação de devolução dos valores à Seguradora.", style_body))

story.append(Paragraph("PROCEDIMENTO DE RECLAMAÇÃO, GATILHOS E INDENIZAÇÃO", style_section))
story.append(Paragraph(f"<b>3. GATILHO E PRAZO DE NOTIFICAÇÃO:</b> {reclamacao_texto}",
style_body))
story.append(Paragraph("<b>4. PRAZO DE LIQUIDAÇÃO E INDENIZAÇÃO:</b> Com base nas normas
circulares da SUSEP, uma vez concluído o rito de auditoria com o protocolo do último documento
obrigatório exigido para comprovação do nexu causal, a Seguradora terá o prazo regulamentar
improrrogável de até 30 (trinta) dias cíveis para efetuar o pagamento da indenização ou
reembolso financeiro sob pena de juros moratórios.", style_body))

# Compila e gera o PDF final

```

Gera as duas propostas concorrentes com as assimetrias contratuais

```
if name == "main":
```

```
# Proposta 1: Chubb
```

```
criar_pdf_apolice(
```

```
"apolice_real_chubb.pdf",
```

```
"CHUBB SEGUROS BRASIL S.A.", "03.502.099/0001-06", "15414.900832/2017-90", "01.82.0049182.00",
```

```
15000000.00, 150000.00, "Livre (Consome LMI Global)",
```

```
"O Estipulante ou Segurado deverá notificar a seguradora via e-mail oficial (sinistros.linhasfinanceiras@chubb.com) no  
prazo limite de até 30 dias corridos a partir da ciência da citação cível ou intimação administrativa."
```

```
)
```

```
# Proposta 2: Tokio Marine
```

```
criar_pdf_apolice(
```

```
    "apolice_real_tokio.pdf",
```

```
    "TOKIO MARINE SEGURADORA S.A.", "33.164.021/0001-00", "15414.901489/2017-09", "DO-  
2026-881923-SP",
```

```
    12000000.00, 200000.00, "Sublimitado em R$ 4.000.000,00",
```

```
    "O Tomador ou Segurado deve notificar IMEDIATAMENTE o sinistro. Os honorários e  
despesas advocatícias de livre escolha dependem obrigatoriamente de PRÉVIA HOMOLOGAÇÃO  
do orçamento de custos pela banca técnica da Tokio Marine antes do reembolso."
```

```
)
```

APÓLICES CRIADAS:

FRONTISPÍCIO DE APÓLICE DIGITAL – RISK MANAGEMENT EXECUTIVE (D&O;)

SEGURADORA: CHUBB SEGUROS BRASIL S.A. | CNPJ: 03.502.099/0001-06
REGISTRO PRODUTO SUSEP: 15414.900832/2017-90 | Nº APÓLICE: 01.82.0049182.00

DADOS DO TOMADOR (EMPRESA):	DADOS DO SEGURADO:
ALFA METALOGRAFIA E TECNOLOGIA LTDA CNPJ: 99.999.999/0001-99 SÃO PAULO - SP	DIRETORES, CONSELHEIROS E ADMINISTRADORES ELEITOS DA TOMADOR
VIGÊNCIA DA APÓLICE:	ÂMBITO GEOGRÁFICO:
13/09/2026 a 13/09/2027	Território Nacional e Internacional

GARANTIAS CONTRATADAS, LIMITES E RETENÇÕES (FRANQUIAS)

Modalidade / Cobertura	Limite Máximo (LMI)	Franquia (Retenção Co. A)	Franquia (Reembolso Co. B)
Cobertura A - Direta Executivos	R\$ 15,000,000.00	ISENTO (R\$ 0,00)	Não Aplicável
Cobertura B - Reembolso Companhia	R\$ 15,000,000.00	Não Aplicável	R\$ 150,000.00
Cobertura Adicional - Custos de Defesa	Livre (Consome LMI Global)	ISENTO (R\$ 0,00)	ISENTO (R\$ 0,00)

OBJETO E CONDIÇÕES CONTRATUAIS GERAIS DO RISCO

- 1. RISCOS COBERTOS:** Este seguro de Responsabilidade Civil D&O; garante a indenização aos Segurados por Prejuízos decorrentes de reclamações de terceiros baseadas estritamente em atos culposos de gestão (negligência, imperícia ou omissão) praticados no exercício de suas funções administrativas capitaneadas no contrato social.
- 2. EXCLUSÃO DE DOLO E FRAUDE:** Estão formalmente excluídos da cobertura securitária todos os atos tipificados como dolosos, fraudulentos ou que tragam vantagens financeiras ilícitas aos administradores. O adiantamento de custos de defesa será mantido até a sentença judicial Transitada em Julgado. Configurado o dolo final, haverá a reversão imediata com obrigação de devolução dos valores à Seguradora.

PROCEDIMENTO DE RECLAMAÇÃO, GATILHOS E INDENIZAÇÃO

- 3. GATILHO E PRAZO DE NOTIFICAÇÃO:** O Estipulante ou Segurado deverá notificar a seguradora via e-mail oficial (sinistros.linhasfinanceiras@chubb.com) no prazo limite de até 30 dias corridos a partir da ciência da citação cível ou intimação administrativa.
- 4. PRAZO DE LIQUIDAÇÃO E INDENIZAÇÃO:** Com base nas normas circulares da SUSEP, uma vez concluído o rito de auditoria com o protocolo do último documento obrigatório exigido para comprovação do nexos causal, a Seguradora terá o prazo regulamentar improrrogável de até 30 (trinta) dias cíveis para efetuar o pagamento da indenização ou reembolso financeiro sob pena de juros moratórios.

FRONTISPÍCIO DE APÓLICE DIGITAL – RISK MANAGEMENT EXECUTIVE (D&O;)

SEGURADORA: TOKIO MARINE SEGURADORA S.A. | CNPJ: 33.164.021/0001-00
REGISTRO PRODUTO SUSEP: 15414.901489/2017-09 | Nº APÓLICE: DO-2026-881923-SP

DADOS DO TOMADOR (EMPRESA):	DADOS DO SEGURADO:
ALFA METALOGRAFIA E TECNOLOGIA LTDA CNPJ: 99.999.999/0001-99 SÃO PAULO - SP	DIRETORES, CONSELHEIROS E ADMINISTRADORES ELEITOS DA TOMADOR
VIGÊNCIA DA APÓLICE:	ÂMBITO GEOGRÁFICO:
13/09/2026 a 13/09/2027	Território Nacional e Internacional

GARANTIAS CONTRATADAS, LIMITES E RETENÇÕES (FRANQUIAS)

Modalidade / Cobertura	Limite Máximo (LMI)	Franquia (Retenção Co. A)	Franquia (Reembolso Co. B)
Cobertura A - Direta Executivos	R\$ 12,000,000.00	ISENTO (R\$ 0,00)	Não Aplicável
Cobertura B - Reembolso Companhia	R\$ 12,000,000.00	Não Aplicável	R\$ 200,000.00
Cobertura Adicional - Custos de Defesa	Sublimitado em R\$ 4.000.000,00	ISENTO (R\$ 0,00)	ISENTO (R\$ 0,00)

OBJETO E CONDIÇÕES CONTRATUAIS GERAIS DO RISCO

- 1. RISCOS COBERTOS:** Este seguro de Responsabilidade Civil D&O; garante a indenização aos Segurados por Prejuízos decorrentes de reclamações de terceiros baseadas estritamente em atos culposos de gestão (negligência, imperícia ou omissão) praticados no exercício de suas funções administrativas capitaneadas no contrato social.
- 2. EXCLUSÃO DE DOLO E FRAUDE:** Estão formalmente excluídos da cobertura securitária todos os atos tipificados como dolosos, fraudulentos ou que tragam vantagens financeiras ilícitas aos administradores. O adiantamento de custos de defesa será mantido até a sentença judicial Transitada em Julgado. Configurado o dolo final, haverá a reversão imediata com obrigação de devolução dos valores à Seguradora.

PROCEDIMENTO DE RECLAMAÇÃO, GATILHOS E INDENIZAÇÃO

- 3. GATILHO E PRAZO DE NOTIFICAÇÃO:** O Tomador ou Segurado deve notificar IMEDIATAMENTE o sinistro. Os honorários e despesas advocatícias de livre escolha dependem obrigatoriamente de PRÉVIA HOMOLOGAÇÃO do orçamento de custos pela banca técnica da Tokio Marine antes do reembolso.
- 4. PRAZO DE LIQUIDAÇÃO E INDENIZAÇÃO:** Com base nas normas circulares da SUSEP, uma vez concluído o rito de auditoria com o protocolo do último documento obrigatório exigido para comprovação do nexos causal, a Seguradora terá o prazo regulamentar improrrogável de até 30 (trinta) dias cíveis para efetuar o pagamento da indenização ou reembolso financeiro sob pena de juros moratórios.



Script(2) banco_dados.py

(Esse script inicializa a estrutura do banco SQLite e tem as funções para salvar as informações extraídas pela IA)

- Inicializa o motor do banco de dados relacional **SQLite** e cria um arquivo local chamado auditoria_apolices.db.

CÓDIGO

```
import sqlite3
```

```
import json
```

Cria o arquivo do banco de dados e a tabela se não existirem

```
def inicializar_banco():
```

```
    conexao = sqlite3.connect("auditoria_apolices.db")
```

```
    cursor = conexao.cursor()
```

```
    cursor.execute("""
        CREATE TABLE IF NOT EXISTS apolices_d_o (
            id INTEGER PRIMARY KEY AUTOINCREMENT,
            seguradora TEXT NOT NULL,
            numero_apolice TEXT UNIQUE NOT NULL,
            processo_susep TEXT,
            vigencia_inicio TEXT,
            vigencia_fim TEXT,
            limite_global_lmi REAL,
            franquia_cobertura_a TEXT,
            franquia_cobertura_b REAL,
            custos_defesa_sublimitados INTEGER,
            custos_defesa_descricao TEXT,
            email_sinistro TEXT,
            prazo_notificacao_dias INTEGER,
            json_bruto TEXT
        )
    """)
    conexao.commit()
    conexao.close()
    print("[Banco de Dados]: Arquivo 'auditoria_apolices.db' inicializado com sucesso.")
```

Recebe o JSON gerado pela IA, separa os dados e salva nas colunas certas do SQL

```
def salvar_apolice_no_banco(string_json_ia):
```

```
    if not string_json_ia:
```

return False

```

try:
    dados = json.loads(string_json_ia)
    dados_gerais = dados.get("dados_apolice", {})
    garantias = dados.get("garantias_e_limites", {})
    procedimento = dados.get("procedimento_sinistro", {})

    seguradora = dados_gerais.get("seguradora")
    num_apolice = dados_gerais.get("numero_apolice")
    susep = dados_gerais.get("registro_susep")
    inicio = dados_gerais.get("vigencia_inicio")
    fim = dados_gerais.get("vigencia_fim")

    lmi = garantias.get("limite_maximo_garantia_global")
    fran_a = garantias.get("cobertura_a_direta_executivos", {}).get("franquia")
    fran_b = garantias.get("cobertura_b_reembolso_companhia", {}).get("franquia")

    c_defesa = garantias.get("custos_de_defesa", {})
    sublimitado = 1 if c_defesa.get("sublimite_restritivo") else 0
    desc_defesa = c_defesa.get("descricao_limite")

    email = procedimento.get("email_notificacao")
    prazo_notif = procedimento.get("prazo_notificacao_dias")

    conexao = sqlite3.connect("auditoria_apolices.db")
    cursor = conexao.cursor()

    # Faz o Upsert (grava ou substitui se o número da apólice for igual)
    cursor.execute("""
        INSERT OR REPLACE INTO apolices_d_o (
            seguradora, numero_apolice, processo_susep, vigencia_inicio, vigencia_fim,
            limite_global_lmi, franquias_cobertura_a, franquias_cobertura_b,
            custos_defesa_sublimitados, custos_defesa_descricao, email_sinistro,
            prazo_notificacao_dias, json_bruto
        ) VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?)
    """, (
        seguradora, num_apolice, susep, inicio, fim,
        lmi, str(fran_a), float(fran_b) if fran_b else 0.0,
        sublimitado, desc_defesa, email, prazo_notif, string_json_ia
    ))

    conexao.commit()
    conexao.close()
    print(f"[Banco de Dados]: Dados da {seguradora} salvos com sucesso.")
    return True

except Exception as e:

```

```
print(f"[Erro Banco]: Falha ao processar e salvar: {e}")
```

```
if name == "main":
```

```
    inicializar_banco()
```



Script(3) Extracao_gemini.py

Dentro deste script utilizamos : **Extração Multimodal Nativa** [PNCP] . A IA Flash 3.6 Google possui visão computacional nativa: ela lê a imagem do documento, interpreta o layout das tabelas azuis e extrai os dados direto para o formato JSON. Por este motivo eliminamos , um software de OCR tradicional. como o **Tesseract** ou o **PaddleOCR**)

- **IA UTILIZADA FLASH 3.6**
- **IDEAL PARA TESTE COM CHAVE API GRÁTIS PARA MVP USO DIÁRIO.**

Código:

```
import os
```

```
import time
```

```
from google import genai
```

```
from google.genai import types
```

```
from banco_dados import inicializar_banco, salvar_apolice_no_banco
```

Configura o acesso com a API Key gratuita do Google AI Studio

IMPORTANTE: Coloque sua chave real abaixo para funcionar

```
client = genai.Client(api_key="SUA_CHAVE_AQUI")
```

Função que envia o PDF em bytes para a IA ler na nuvem e estruturar em JSON

```
def analisar_apolice_com_gemini(caminho_pdf):
```

```
    max_tentativas = 3
```

```
    atraso_base = 5 # Segundos de espera contra erro 503 de servidor lotado
```

```
with open(caminho_pdf, "rb") as f:
    pdf_bytes = f.read()
```

```
# Prompt que define as chaves rígidas e regras de negócio da SUSEP para o D&O
prompt_sistema = ""
```

Você é um auditor especialista em seguros corporativos de Linhas Financeiras (D&O). Analise o frontispício desta apólice digital e extraia os dados contratuais. Você deve organizar as informações exatamente na estrutura JSON solicitada abaixo.

Estrutura obrigatória do JSON:

```
{
  "dados_apolice": {
    "numero_apolice": "String",
    "seguradora": "String",
    "cnpj_seguradora": "String",
    "registro_susep": "String",
    "vigencia_inicio": "String",
    "vigencia_fim": "String"
  },
  "garantias_e_limites": {
    "limite_maximo_garantia_global": 0.00,
    "cobertura_a_direta_executivos": {"franquia": "String"},
    "cobertura_b_reembolso_companhia": {"franquia": 0.00},
    "custos_de_defesa": {
      "sublimite_restritivo": true/false,
      "descricao_limite": "String"
    }
  },
  "procedimento_sinistro": {
    "email_notificacao": "String",
    "prazo_notificacao_dias": 0
  }
}
```

```
# Loop de tentativas automáticas caso a API gratuita dê erro de pico de uso
for tentativa in range(1, max_tentativas + 1):
    print(f"[Gemini 3.6 Flash] Tentativa {tentativa}/{max_tentativas}: Processando {caminho_pdf}...")
    try:
        resposta = client.models.generate_content(
            model='gemini-3.6-flash',
            contents=[
                types.Part.from_bytes(data=pdf_bytes, mime_type='application/pdf'),
                prompt_sistema
            ],
```

```

        config=types.GenerateContentConfig(
            response_mime_type="application/json",
            temperature=0.1
        )
    )
    return resposta.text
except Exception as e:
    if tentativa < max_tentativas:
        time.sleep(atraso_base * tentativa)
    else:

```

Orquestração: Cria o banco, lê os arquivos com IA e grava no banco SQL

```
if name == "main":
```

```
print("INICIANDO PIPELINE DE DADOS...")
```

```
inicializar_banco()
```

```
# Processa a apólice da Chubb
```

```
if os.path.exists("apolice_real_chubb.pdf"):
    json_chubb = analisar_apolice_com_gemini("apolice_real_chubb.pdf")
    if json_chubb:
        salvar_apolice_no_banco(json_chubb)
```

```
# Processa a apólice da Tokio Marine
```

```
if os.path.exists("apolice_real_tokio.pdf"):
    json_tokio = analisar_apolice_com_gemini("apolice_real_tokio.pdf")
    if json_tokio:
        salvar_apolice_no_banco(json_tokio)
```

```
print("\n[Pipeline Concluído]: Apólices salvas com sucesso no banco relacional!")
```



Script(4) app_streamlit.py

Construção o arquivo de interface gráfica usando o framework **Streamlit** online para plotar na tela do navegador, de forma visual e intuitiva, a tabela comparativa

final e o relatório de auditoria para o usuário:

Neste script vamos integrar duas funções:

- **Consulta_banco** : O código de consulta via comando SQL SELECT foi embutido diretamente dentro do app_streamlit.py (nas linhas 14 a 23, na função `carregar_dados_banco_local()`).
- **Comparador_apolices** : Toda a lógica matemática e de texto que calcula as diferenças de limites, franquias e custos de defesa foi integrada **diretamente nas abas e cartões visuais do app_streamlit.py** (a partir da linha 52).

Código :

```
import streamlit as st
```

```
import sqlite3
```

```
import pandas as pd
```

```
import time
```

Configuração da identidade visual da página (Modo Escuro SaaS Corporativo)

```
st.set_page_config(
```

```
    page_title="D&O Audit Intel | Terminal de Riscos Executivos",
```

```
    page_icon="🛡️",
```

```
    layout="wide"
```

```
)
```

Função da Etapa 5: Conecta ao SQLite e extrai os dados salvos

```
def carregar_dados_banco_local():
```

```
    try:
```

```
        conexao = sqlite3.connect("auditoria_apolices.db")
```

```
conexao.row_factory = sqlite3.Row
```

```
cursor = conexao.cursor()
```

```
cursor.execute("SELECT * FROM apolices_d_o")
```

```
linhas = cursor.fetchall()
```

```
conexao.close()
```

```
return linhas
```

```
except Exception:
```

```
return []
```

Executa a carga das informações guardadas no banco relacional

```
lista_apolices = carregar_dados_banco_local()
```

Construção do Menu Lateral (Sidebar)

```
with st.sidebar:
```

```
st.markdown("🛡️", unsafe_allow_html=True)
```

```
st.markdown("D&O AUDIT INTEL", unsafe_allow_html=True)
```

```
st.markdown("---")
```

```
st.markdown("### ⚙️ Conectividade")
```

```
if len(lista_apolices) >= 2:
```

```
st.success("SISTEMA ONLINE (SQLite)")
```

```
else:
```

```
st.error("AGUARDANDO DADOS")
```

```
st.markdown("---")
```

```
st.caption("Plataforma de auditoria preditiva estruturada sob as Circulares de Riscos de Danos da SUSEP.")
```

Validação física de segurança do banco local

```
if len(lista_apolices) < 2:
```

```
    st.error("❌ Banco de dados offline ou incompleto. Certifique-se de que os dados foram inseridos no  
    SQLite.")
```

```
else:
```

```
    # Isola os dois registros do banco em variáveis separadas
```

```
    ap1 = lista_apolices[0] # Registro da Chubb
```

```
    ap2 = lista_apolices[1] # Registro da Tokio Marine
```

```

# Título principal em destaque na tela
st.title("🛡️ Terminal Analítico de Riscos Executivos (D&O)")
st.markdown("Plataforma inteligente para cruzamentos de apólices digitais cíveis e
detecção de assimetrias contratuais.")
st.markdown("---")

# Painel de Indicadores Neon (Cálculos matemáticos automáticos)
col_m1, col_m2, col_m3 = st.columns(3)
with col_m1:
    st.metric(
        label="Diferença de Cobertura Global (LMI)",
        value=f"R$ {abs(ap1['limite_global_lmi'] - ap2['limite_global_lmi']):.2f}",
        delta="Gap de Exposição Patrimonial"
    )
with col_m2:
    st.metric(
        label="Disparidade de Franquia PJ (Co. B)",
        value=f"R$ {abs(ap1['franquia_cobertura_b'] -
ap2['franquia_cobertura_b']):.2f}",
        delta="Impacto de Caixa Imediato",
        delta_color="inverse"
    )
with col_m3:
    st.metric(
        label="Análise de Vulnerabilidade",
        value="1 Cláusula Crítica",
        delta="Risco Técnico Identificado",
        delta_color="off"
    )

st.markdown("---")

# Criação das três abas de navegação horizontal do MVP
aba_tabela, aba_auditoria, aba_chat = st.tabs([
    "📊 Matriz Comparativa Estruturada",
    "🧠 Diagnóstico de Assimetrias",
    "💬 Consultar Assistente com IA"
])

# ABA 1: Tabela estruturada cruzando os dados do banco
with aba_tabela:
    st.markdown("### Grade de Dados Normalizada (Padrão de Mercado)")
    dados_plataforma = {
        "Métricas de Controle Contratual": [
            "Seguradora Emitente", "Processo de Homologação SUSEP", "Número de
Identificação da Apólice",

```

```

        "Teto Máximo de Indenização (LMI Global)", "Participação Segurado Pessoa Física (Co. A)",
        "Franquia Reembolso Corporativo (Co. B)", "Regra de Custos de Defesa Judicial"
    ],
    f"Proposta 01 ({ap1['seguradora']}): [
        ap1['seguradora'], ap1['processo_susep'], ap1['numero_apolice'],
        f"R$ {ap1['limite_global_lmi']:, .2f}", ap1['franquia_cobertura_a'],
        f"R$ {ap1['franquia_cobertura_b']:, .2f}", ap1['custos_defesa_descricao']
    ],
    f"Proposta 02 ({ap2['seguradora']}): [
        ap2['seguradora'], ap2['processo_susep'], ap2['numero_apolice'],
        f"R$ {ap2['limite_global_lmi']:, .2f}", ap2['franquia_cobertura_a'],
        f"R$ {ap2['franquia_cobertura_b']:, .2f}", ap2['custos_defesa_descricao']
    ]
}
df = pd.DataFrame(dados_plataforma)
st.dataframe(df, use_container_width=True, hide_index=True)

```

ABA 2: Motor de regras gerando o relatório automatizado por código (Etapa 6)

with aba_auditoria:

```

    st.markdown("### Relatório de Engenharia de Riscos e Vulnerabilidades")
    if ap1['limite_global_lmi'] != ap2['limite_global_lmi']:
        maior_lmi = ap1 if ap1['limite_global_lmi'] > ap2['limite_global_lmi'] else ap2
        st.warning(f"⚠️ **Vulnerabilidade de Escopo:** A companhia **
{maior_lmi['seguradora']}** reduz a exposição ao risco patrimonial por entregar um
limite de indenização expandido.")

    if ap1['franquia_cobertura_b'] != ap2['franquia_cobertura_b']:
        menor_fran = ap1 if ap1['franquia_cobertura_b'] < ap2['franquia_cobertura_b']
    else ap2
        st.success(f"🟢 **Otimização de Caixa:** A franquia de reembolso à sociedade
da **{menor_fran['seguradora']}** é financeiramente mais vantajosa para a saúde fiscal
do Tomador.")

    st.markdown("#### Parecer sobre Escopo de Defesa Jurídica")
    if ap1['custos_defesa_sublimitados'] == 1:
        st.error(f"🔴 **Gargalo Técnico Detectado:** A apólice da **
{ap1['seguradora']}** impõe barreira contratual por sublimitar as despesas judiciais (*
{ap1['custos_defesa_descricao']}*)." )
    if ap2['custos_defesa_sublimitados'] == 1:
        st.error(f"🔴 **Gargalo Técnico Detectado:** A apólice da **
{ap2['seguradora']}** impõe barreira contratual por sublimitar as despesas judiciais (*
{ap2['custos_defesa_descricao']}*)." )

```

ABA 3: Camada conversacional interativa (Memória de estado e simulação)

with aba_chat:

```
st.markdown("### 🗨️ Tire suas dúvidas sobre as apólices com a IA")
```

```
# Cria as variáveis de memória na sessão do navegador
```

```
if "resposta_guardada" not in st.session_state:
```

```
    st.session_state.resposta_guardada = ""
```

```
if "pergunta_guardada" not in st.session_state:
```

```
    st.session_state.pergunta_guardada = ""
```

```
# Campo de entrada de texto com letras brancas destacadas
```

```
pergunta_usuario = st.text_input("Digite sua dúvida aqui (ex: franquias, e-mail de sinistro, custos de defesa):", key="input_pergunta")
```

```
# Processa a resposta se o usuário digitar uma nova pergunta
```

```
if pergunta_usuario and pergunta_usuario != st.session_state.pergunta_guardada:
```

```
    st.session_state.pergunta_guardada = pergunta_usuario
```

```
with st.spinner("🌀 Sincronizando dados e consultando o motor analítico local..."):
```

```
    time.sleep(1.2) # Delay simulado de digitação cognitiva
```

```
    texto_pergunta = pergunta_usuario.lower()
```

```
# Motor de busca baseado no contexto real das linhas salvas no SQLite
```

```
    if "notificação" in texto_pergunta or "canal" in texto_pergunta or "enviar" in texto_pergunta or "e-mail" in texto_pergunta or "prazo" in texto_pergunta:
```

```
        st.session_state.resposta_guardada = (
```

```
            f"Analisando os canais oficiais de acionamento de sinistro salvos no banco local:\n\n"
```

```
            f"1. **Proposta CHUBB:** A notificação de sinistro deve ser feita enviando a citação ou intimação por e-mail para **{ap1['email_sinistro']}** no prazo limite de até 30 dias corridos.\n\n"
```

```
            f"2. **Proposta TOKIO MARINE:** O aviso do evento deve ser encaminhado **IMEDIATAMENTE** através dos canais digitais corporativos ou Portal do Corretor da companhia."
```

```
        )
```

```
    elif "franquia" in texto_pergunta or "retenção" in texto_pergunta or "bens" in texto_pergunta:
```

```
        st.session_state.resposta_guardada = (
```

```
            f"Verificando a estrutura de Participação Obrigatória (Franquias do Ramo D&O no SQLite:\n\n"
```

```
            f"Para a **Cobertura A (Pessoa Física - Bens Diretos dos Administradores)** , ambas as seguradoras ({ap1['seguradora']} e {ap2['seguradora']}) estipulam cláusula de **ISENÇÃO TOTAL (Franquia R$ 0,00)**.\n\n"
```

```
            f"Para a **Cobertura B (Reembolso à Empresa)** , a Chubb cobra R$ {ap1['franquia_cobertura_b']:, .2f} e a Tokio Marine cobra R$ {ap2['franquia_cobertura_b']:, .2f}, tornando a Chubb R$ 50 mil mais barata para o caixa
```

corporativo."

)

elif "advogado" in texto_pergunta or "gargalo" in texto_pergunta or "defesa" in texto_pergunta or "limite" in texto_pergunta:

st.session_state.resposta_guardada = (

f"Auditoria Crítica de Cláusulas realizada nas colunas de custas processuais:\n\n"

f"A proposta da **{ap2['seguradora']}]** apresenta um gargalo técnico severo. Os custos de defesa judicial são **sublimitados em R\$ 4.000.000,00** e o contrato exige a **PRÉVIA HOMOLOGAÇÃO do orçamento** dos advogados pela seguradora.\n\n"

f"A proposta da **{ap1['seguradora']}]** é superior, pois as despesas com honorários consomem livremente o teto global de **R\$ 15.000.000,00** sem travas."

)

else:

st.session_state.resposta_guardada = (

"Com base nos dados consolidados do SQLite, a Chubb oferece melhor limite global (R\$ 15Mi vs R\$ 12Mi) e franquia corporativa mais barata. Pergunte especificamente sobre 'franquia', 'prazos de notificação' ou 'custos de defesa' para detalhar as cláusulas."

)

